

Optimizing Lightweight Architectures for Fine-Grained Vehicle Body Type Classification Under Strict Deployment Constraints

The deployment of deep learning models in edge computing environments or highly constrained academic and enterprise servers necessitates a delicate equilibrium between predictive accuracy, computational latency, and overall payload size. In the specific context of classifying fine-grained vehicle body types across eight distinct and highly specific categories—SUV, VAN, STATION WAGON, MICRO, AÇIK TEKERLEKLİ (Formula 1), SEDAN, HATCHBACK, and PICK UP—the challenge is significantly amplified by overlapping morphological geometries and the presence of severe domain bias in uncured datasets.

The project parameters impose a stringent requirement: the complete application payload, encompassing the trained model weights, the Python backend infrastructure, the compiled web frontend, and all necessary runtime dependencies, must not exceed a maximum storage footprint of 95 MB. Furthermore, the application must demonstrate exceptional generalization capabilities on a completely unseen test dataset. Finally, the system must achieve near real-time inference latency on standard CPU hardware during live deployment, prohibiting the reliance on cloud-based GPU acceleration or massive deep neural networks.

Meeting these demanding requirements requires a radical departure from traditional, heavy-weight deep learning paradigms. Standard approaches utilizing massive Vision Transformers (ViTs) or deep Convolutional Neural Networks (CNNs) like ResNet-152 are immediately disqualified by their sheer parameter counts, which often translate to file sizes well over 100 MB and sluggish CPU execution times. Instead, the solution must be engineered through a multi-faceted, highly optimized approach. This encompasses aggressive dataset curation to eliminate background dependency, the deployment of state-of-the-art structurally reparameterized lightweight architectures, targeted loss function formulation to resolve fine-grained morphological confusion, and a meticulously crafted full-stack web architecture utilizing a headless inference engine connected to a minified React frontend.

Strategic Data Acquisition and Taxonomy Construction

The foundation of robust generalization on an unseen test dataset is a comprehensive, highly diverse, and perfectly balanced training corpus. Standard vehicle classification datasets frequently lack niche categories, rendering them insufficient for an eight-class problem that includes specialized racing vehicles and ultra-compact urban cars. For example, the widely

utilized Stanford Cars dataset contains 16,185 images across 196 classes at the make and model level, but it conspicuously lacks both Formula 1 vehicles and modern microcars.¹ Therefore, these missing classes must be systematically sourced, filtered, and seamlessly integrated without introducing structural anomalies or domain biases into the training pipeline.

Sourcing the "Açık Tekerlekli" (Formula 1) Class

Formula 1 (F1) vehicles are visually distinct, characterized by open-wheel configurations, massive aerodynamic appendages (front and rear wings), an elongated central monocoque, and a lack of traditional passenger cabins or windshields. However, they are notoriously difficult to source in balanced quantities compared to ubiquitous passenger vehicles like Sedans or SUVs. To fill the F1 class gap cleanly, several open-source repositories provide highly curated image sets that have been scraped and verified by the community.

The Kaggle data ecosystem offers several optimal sources for this specific class. The "Formula One cars" dataset provides images strictly categorized by their corresponding F1 racing teams, ensuring a diversity of liveries and chassis designs.² More comprehensively, the "f1-image-classification-updated" repository contains thousands of high-resolution images of modern F1 chassis types organized into directories such as Mercedes, Red Bull Racing, Ferrari, McLaren, and AlphaTauri.⁴ By aggregating these sources, a robust sub-dataset of thousands of F1 images can be constructed. It is vital to ensure that images selected depict the cars from multiple viewpoints (front, profile, isometric) rather than relying solely on extreme telephoto tracking shots common in sports photography, which can distort the vehicle's true geometry.

Sourcing the "Micro" Car Class

The "Micro" car class presents a completely different taxonomic challenge, as its definition varies regionally and overlaps with several other subcompact categories. Microcars are generally characterized by extreme compactness, designed primarily for high-density urban mobility. This category heavily overlaps with the Japanese "Kei car" (Keijidosha) classification. Established in the aftermath of World War II to stimulate the Japanese economy, Kei cars are strictly regulated vehicles that, in their modern iteration, are legally restricted to a maximum engine displacement of 660cc, a maximum length of 3.4 meters (11.2 feet), and a width of 1.48 meters (4.9 feet).⁵ Because of these strict dimensional constraints, Kei cars and Microcars exhibit a highly specific visual geometry: their wheels are pushed to the absolute extreme corners of the chassis to maximize interior cabin space, resulting in virtually nonexistent front and rear overhangs.⁶

To source high-quality Micro and Kei car imagery, standard datasets are insufficient. Instead, the HuggingFace repository Unit293/car_models_3887 is highly recommended.⁸ This massive dataset, scraped entirely from Autoevolution, contains over 193,000 images across 3,778 car model variants.⁸ Most importantly, it includes an exhaustive CSV file containing 44 columns of metadata, including "body type".⁸ By filtering this metadata for models historically classified as Micro or Kei cars—such as the Fiat 500 (Cinquecento), Smart Fortwo, Honda N Sedan, Suzuki Alto, Daihatsu Copen, and Mitsubishi Minica—a highly accurate and diverse sub-dataset can be

assembled.⁶

Furthermore, the HuggingFace dataset DamianBoborzi/car_images provides a highly curated, pre-processed subset of these Autoevolution images. In this dataset, the vehicles have already been detected using YOLOv11, and the background has been algorithmically removed using rembg, followed by a square crop and aesthetic filtering.¹² Leveraging this pre-processed dataset is an incredibly powerful shortcut for acquiring Micro car data that is already primed for bias-free training.

Dataset Source	Target Class	Image Volume	Metadata/Features	Recommended Use
Stanford Cars	General (Sedan, SUV, Pickup)	16,185	Make, Model, Year	Baseline training for standard passenger vehicles. ¹
f1-image-classification	Açık Tekerlekli (F1)	~6,000+	Team labels (Ferrari, Mercedes)	Primary source for open-wheel racing chassis. ⁴
Unit293/car_models_3887	Micro / Kei Cars	193,000	44 metadata columns (Body Type)	Querying CSV for specific Micro models (Fiat 500, Suzuki Alto). ⁸
DamianBoborzi/car_images	Micro / General	High	Backgrounds removed via YOLOv11	Ideal for eliminating domain bias during fine-tuning. ¹²

Eradicating Domain and Background Bias

A critical failure mode in fine-grained image classification occurs when a deep learning model learns contextual background features rather than the morphological characteristics of the target object. This phenomenon, known in the machine learning literature as "shortcut learning" or "selection bias," leads to severe and catastrophic generalization degradation on unseen

data.¹³

In this specific eight-class project, domain bias is arguably the single highest risk factor. Because standard data scraping relies on organic photography, specific vehicle classes are inextricably linked to specific environments. Formula 1 cars are predominantly photographed on paved racetracks with asphalt, curbs, grandstands, and catch-fencing in the background. Conversely, SUVs and Pick Up trucks are frequently depicted in off-road, mountainous, dirt, or rugged environments to emphasize their utility.¹³ If this bias is left unmitigated, the neural network will optimize its loss function by looking at the easiest available features: the background. It will learn to classify "dirt roads" as "SUV" and "asphalt with red and white curbs" as "F1." When presented with an unseen test image of an F1 car in a museum, or an SUV parked in a paved urban driveway, the model will fail completely.

Algorithmic Background Subtraction via U2NetP

To ensure the model learns the vehicle's body type rather than its environment, a two-stage preprocessing and augmentation strategy is required. The most definitive method to eliminate background bias is explicit background subtraction. By generating highly accurate alpha mattes and extracting the vehicle from its background, we guarantee that the classification network only processes the pixels belonging to the car itself.

For a strictly constrained 95 MB project, utilizing massive segmentation models like Mask R-CNN or SAM (Segment Anything Model) within the final deployment is unviable. However, the background removal can be performed entirely offline during the dataset preparation phase. Even if real-time background removal is desired in the final pipeline, highly efficient models exist. The U2Net architecture—a two-level nested U-structure network specifically designed for salient object detection (SOD)—is ideal for this task. While the standard U2Net model file is 176.3 MB, its lightweight variant, U2NetP, boasts a model file size of exactly 4.7 MB.¹⁵

U2NetP achieves this remarkably small footprint by reducing the channel dimensions in the nested U-blocks, yet it retains the ability to extract high-resolution salient object masks from complex scenes without prior human input.¹⁷ Integrating U2NetP into the data preprocessing pipeline allows for the rapid, automated generation of binary masks. By applying these masks, the original backgrounds can be completely removed. Replacing the backgrounds with uniform white, solid gray, or heavily randomized Perlin noise forces the downstream classification network to rely entirely on the structural edges, panel gaps, glasshouse shapes, and wheel arches of the vehicle.¹²

Advanced Augmentation and Attention Mechanisms

If offline background subtraction is deemed too aggressive, or if the unseen test dataset contains highly complex lighting that interacts with the background, advanced data augmentation must be heavily utilized. Techniques such as randomized background swapping (where a vehicle is artificially pasted onto entirely uncorrelated backgrounds, such as an F1 car on a dirt road) actively penalize the network for relying on contextual clues.

Furthermore, recent research in fine-grained car recognition has demonstrated the efficacy of architectural enhancements like the Hybrid Attention Module (HAM). When integrated into lightweight backbones, a HAM forces the network model to focus its receptive field more strictly on target objects with consistent classes, actively suppressing task-irrelevant backgrounds and other noise.¹⁹ This attention mechanism acts as a regularizer, ensuring that the gradient updates are driven by the pixels corresponding to the car's body panels rather than the surrounding environment, significantly improving the learning ability and generalization of the network on unseen data.¹⁹

Selecting and Benchmarking Ultra-Lightweight Network Architectures

Maximizing the macro F1-score while adhering to a strict 95 MB full-project storage budget and a low CPU latency constraint requires the selection of a highly optimized deep learning backbone. Traditional Convolutional Neural Networks like VGG-16, ResNet-50, or DenseNet-121 exceed the 95 MB limit on their own (ResNet-50 typically requires ~98 MB for weights alone) and perform poorly on raw CPU hardware without specialized GPU or TPU accelerators.²⁰

The chosen architecture must natively support efficient spatial downsampling, minimize floating-point operations per second (FLOPs), and restrict the total parameter count to roughly under 10 million. This ensures the model file remains under 30-40 MB in standard FP32 precision, leaving ample space within the 95 MB budget for the Python runtime environment, frontend build files, and Gradio dependencies.

The MobileNet Paradigm: V3 and V4

The MobileNet family, developed by Google, represents the de facto standard for resource-constrained computer vision tasks. MobileNetV3 utilizes depthwise separable convolutions, inverted residual blocks, and lightweight Squeeze-and-Excite (SE) modules to achieve high accuracy with minimal computational overhead.²² The MobileNetV3-Large variant operates with approximately 6.4 million parameters, resulting in a raw FP32 file size of around 25 to 30 MB, and achieves an ImageNet Top-1 accuracy of approximately 86.2%.²¹

However, recent advancements have yielded MobileNetV4, a generational leap in mobile architecture. MobileNetV4 introduces the Universal Inverted Bottleneck (UIB) block, which unifies previous bottleneck designs into a single, highly adaptable module.²⁵ Furthermore, it incorporates Mobile MQA (Multi-Query Attention) to selectively apply self-attention without the massive memory bandwidth bottlenecks typical of traditional transformers.²⁶ MobileNetV4 provides up to a 75% reduction in parameter utilization compared to standard models and executes 3-4 times faster on CPU hardware than previous iterations.²⁶ A quantized MobileNetV4 model file can weigh as little as 33.3 MB while processing over 290 queries per second (QPS) on mobile CPUs, making it an extraordinarily formidable candidate for this project.²⁸

Hybrid Architectures: MobileViT and FastViT

While CNNs excel at extracting local inductive biases (edges, textures), Vision Transformers (ViTs) are superior at capturing global contextual representations. Pure ViTs are generally too heavy and computationally expensive for this project's constraints.²² However, hybrid architectures that merge CNN feature extraction with ViT self-attention mechanisms provide an optimal middle ground.

MobileViT achieves this by presenting a different perspective on global processing: treating transformers as convolutions.²² It achieves 78.4% Top-1 ImageNet accuracy with roughly 6 million parameters, significantly outperforming MobileNetV3 at a similar parameter count while remaining lightweight enough for CPU deployment.²²

Even more suitable for the absolute strict CPU latency requirements of a live presentation is FastViT. FastViT is a hybrid vision transformer that employs "structural reparameterization".²⁹ During training, the network utilizes complex branching and skip connections to facilitate gradient flow and robust feature learning. However, during export for inference, these complex branches are mathematically folded (reparameterized) into a single sequential convolution path.³⁰ This eliminates the memory access costs associated with skip connections during inference. FastViT is aggressively optimized for standard CPUs and Apple Neural Engines, providing processing speeds that are up to 1.9x faster than optimized ConvNeXt models and 26% faster than MobileOne, all while maintaining a parameter count of roughly 6.8 million (FastViT-S12).²⁹

Architecture	Approximate Parameters	Unquantized Size (FP32)	CPU Latency Profile	Core Architectural Innovation
MobileNetV3-Large	6.4 Million	~25 MB	Very Fast	Hard-Swish, Squeeze-and-Excite ²⁴
MobileNetV4-Conv	5-7 Million	~20-30 MB	Ultra Fast	Universal Inverted Bottleneck (UIB) ²⁵
MobileViT-S	6.9 Million	~27 MB	Moderate	Transformer treated as Convolution ²²

FastViT-S12	6.8 Million	~27 MB	Ultra Fast	Structural Reparameterization ³⁰
--------------------	-------------	--------	------------	---

Given the absolute necessity for low CPU latency during the live presentation, and the strict file size limit, **FastViT-S12** or **MobileNetV4-Conv** are the premier recommendations. Their unquantized sizes remain comfortably beneath the 40 MB threshold, leaving over 50 MB for the frontend, Python dependencies, and backend logic, while guaranteeing state-of-the-art inference speeds.

Precision Quantization and CPU Acceleration via ONNX

To guarantee near-instantaneous predictions during a live presentation on standard CPU hardware, the trained PyTorch (.pt) or TensorFlow (.h5) model must be exported and heavily optimized. While raw FP32 models might technically fit within the 95 MB storage budget, retaining the PyTorch or TensorFlow libraries as project dependencies will instantly violate the size constraint (PyTorch alone often exceeds 700 MB). Therefore, converting the model to the ONNX (Open Neural Network Exchange) format and applying quantization is an absolute necessity.

The Superiority of the ONNX Runtime

Exporting the model to ONNX decouples the model architecture from the heavy deep learning training libraries. This allows the final application to run inference using onnxruntime, a highly optimized, cross-platform C++ backend with a very lightweight Python wrapper.³² The onnxruntime library is specifically designed for high-performance inference, leveraging CPU-specific vectorization instructions (such as AVX2 or AVX-512) to accelerate matrix multiplications significantly.³³ Dropping the PyTorch dependency and relying solely on onnxruntime can save hundreds of megabytes in the deployment virtual environment, ensuring the entire project folder remains strictly under the 95 MB limit.³⁴

Integer Quantization Techniques and MobileNet Pitfalls

Quantization involves systematically reducing the numerical precision of the network's internal weights and activations from 32-bit floating-point (FP32) decimals to 8-bit integers (INT8). This mathematical process typically reduces the model's physical file size by a factor of four (e.g., shrinking a 28 MB FastViT model down to approximately 7 MB) and vastly accelerates CPU inference by replacing complex floating-point arithmetic with rapid, low-power integer math.³³

There are two primary methods for applying post-training quantization (PTQ) via the onnxruntime.quantization API:

1. **Dynamic Quantization (quantize_dynamic):** The weights of the model are pre-quantized from FP32 to INT8 offline. However, during live inference, the activation

values are dynamically quantized to INT8 at runtime based on their observed range for that specific image.³² This requires no calibration dataset and is highly robust.

2. **Static Quantization (quantize_static):** Both the weights and the expected activations are quantized ahead of time. This requires passing a representative "calibration dataset" through the model so it can observe the minimum and maximum activation ranges and lock in the scale and zero-point factors.³² This offers the fastest possible inference speed but is highly susceptible to catastrophic accuracy loss if the calibration is flawed.

Applying INT8 post-training static quantization to MobileNet and FastViT architectures carries a well-documented and severe risk. The depthwise separable convolutions that give these models their efficiency tend to produce weight distributions with extreme outliers and highly skewed activation ranges. Specifically, when using static quantization on MobileNetV3 via ONNX, engineers frequently encounter massive accuracy degradation and mathematical overflow errors. The F1-score plummets, and variables such as INT_MIN or -2147483648 appear in the quantized bias tensors.³⁸ This mathematical collapse occurs because the convolution weights in a pretrained MobileNet are incredibly small; applying a standard linear quantization scale factor causes massive underflow, destroying the feature maps.³⁸

Actionable Quantization Strategy

To mitigate this mathematical collapse while maintaining blazing fast CPU inference, the following protocol should be strictly observed:

1. **Quantization-Aware Training (QAT):** If compute resources permit, utilize QAT rather than PTQ. QAT simulates the effects of low-precision INT8 arithmetic during the final fine-tuning phase, allowing the network to organically adapt its weights to survive the precision drop, thus preventing catastrophic accuracy drops entirely.³⁶
2. **Dynamic Quantization over Static:** If QAT is unfeasible, utilize **Dynamic Quantization** exclusively. While it introduces a microscopic overhead to calculate activation scales dynamically at runtime, it is significantly more robust against the activation outliers present in MobileNet and FastViT architectures. It ensures the F1-Score remains fully intact while still cutting the physical model size by up to 70%.³²
3. **Per-Channel Weighting:** When executing the ONNX quantization script, always enable `per_channel=True` for the weight quantization. This calculates an independent scale and zero-point for each output channel of the convolution, successfully accommodating the extreme variance inherent in depthwise layers.³⁸

Python

```
# python backend snippet: Exporting and Quantizing
```



```

import torch
import onnx
from onnxruntime.quantization import quantize_dynamic, QuantType

# 1. Export PyTorch model to ONNX
dummy_input = torch.randn(1, 3, 224, 224)
torch.onnx.export(model, dummy_input, "model_fp32.onnx",
                  opset_version=13, input_names=['input'], output_names=['output'])

# 2. Apply Dynamic Quantization to fix MobileNet overflow issues
quantize_dynamic(
    model_input="model_fp32.onnx",
    model_output="model_int8.onnx",
    per_channel=True, # Critical for depthwise convolutions
    weight_type=QuantType.QUInt8
)
# The resulting "model_int8.onnx" will be ~7MB and highly performant on CPUs.

```

Overcoming Fine-Grained Morphological Confusion

The classification of vehicles into eight distinct body types is inherently a fine-grained image classification problem. Unlike coarse-grained classification (e.g., distinguishing a car from a dog), fine-grained tasks require the network to discern minute, localized feature disparities. Several classes within this specific project share massive morphological overlaps. The model will almost certainly exhibit extreme confusion between "Station Wagon" and "SUV", as well as "Hatchback" and "Micro". Addressing this requires a nuanced understanding of their visual differences and the application of advanced loss formulations during training.

Identifying Crucial Visual Distinctions

To force the neural network to distinguish between visually adjacent classes, the training data and architecture must emphasize specific structural geometries.

- **Station Wagon vs. SUV:** A Station Wagon is fundamentally a passenger car chassis with an elongated roofline extending rearward to maximize cargo space. It maintains a low, car-like ride height, low ground clearance, and an aerodynamic front fascia. Conversely, an SUV (Sport Utility Vehicle) is characterized by a significantly taller height profile, elevated ground clearance (often utilizing a truck-based or robust unibody platform), and a taller, bulkier roof intended for vertical cargo.⁴⁰ The model must learn to index the vertical distance between the wheel arches and the ground, as well as the aspect ratio of the greenhouse (the glassed-in upper section of the vehicle body).
- **Hatchback vs. Micro:** Both body types feature a truncated rear end with a door that swings upward. However, a Hatchback is built on a standard subcompact or compact chassis with a distinct front hood. A Micro car is defined by extreme geometric

proportions. Micro cars feature an ultra-short wheelbase, wheels pushed to the absolute corners of the chassis, and an almost entirely vertical rear hatch with a very stubby front hood.⁷

Advanced Loss Formulations: Focal Loss and Label Smoothing

Standard Cross-Entropy loss functions perform poorly in fine-grained tasks because they treat all misclassifications equally and allow the network to become overconfident in its predictions on "easy" classes, ignoring the subtle boundaries between "hard" classes. To maximize the macro F1-score and penalize confusion between similar body types, two advanced loss strategies must be integrated.

Focal Loss Implementation Focal Loss was explicitly designed to address severe class imbalance and force neural networks to focus on "hard" examples. By adding a modulating factor $(1 - p_t)^\gamma$ to the standard cross-entropy criterion, the loss contributed by easy, highly confident examples is dynamically down-weighted.⁴⁴

Mathematically, if the network easily distinguishes F1 cars from Pick Up trucks (assigning a high probability p_t to the correct class), the modulating factor $(1 - p_t)^\gamma$ approaches zero, meaning the network stops wasting gradient updates on these obvious differences. Meanwhile, if the network continually struggles to separate Hatchbacks from Micro cars (where p_t is low), the modulating factor remains near 1.0, ensuring these "hard" examples dominate the backpropagation process. Setting the focusing parameter $\gamma = 2$ and incorporating an α balancing parameter based on the exact class frequencies of the scraped dataset will significantly elevate the macro F1-score across the heavily confused classes.⁴⁴

Knowledge-Distillation-Based Label Smoothing Traditional Label Smoothing (LS) regularizes the network by converting hard "one-hot" target vectors (e.g., [1, 0, 0, 0...]) into soft targets (e.g., [0.9, 0.014, 0.014...]), distributing a small amount of probability mass uniformly across all incorrect classes. While this prevents the network from becoming overly confident, recent research indicates that uniform LS can degrade the model's ability to selectively classify, as it entirely ignores the actual semantic and visual similarity between specific classes.⁴⁷

Instead, **Knowledge-Distillation-Based Label Smoothing** should be utilized. Rather than distributing the smoothing value α uniformly, the probability mass is distributed proportionally based on the visual and semantic distance between classes.⁴⁸ In this paradigm, the network is allowed to acknowledge that a Station Wagon is visually much closer to an SUV than it is to an F1 car. By incorporating actual class confusion probabilities (derived from a teacher model or a confusion matrix) into the target labels, the model's calibration improves drastically. It reduces overconfidence-induced overfitting without arbitrarily penalizing the network for recognizing legitimate structural similarities between adjacent body types.⁴⁸

Designing the Optimized Full-Stack Architecture

The final hurdle is delivering a modern, beautiful web interface using a React frontend and a Gradio/FastAPI backend, while strictly adhering to the 95 MB size constraint and ensuring low latency. Traditional methods of connecting independent frontend and backend servers require complex deployment environments, massive `node_modules` folders, and heavy API libraries, which would instantly violate the university upload constraints.

The optimal, modern solution is a unified, "headless" Gradio architecture served entirely through FastAPI, coupled with an aggressively optimized React Single Page Application (SPA). This clean separation allows the frontend to focus exclusively on user interactions while the backend handles the complex ONNX processing logic.⁴⁹

The Headless Gradio Backend (FastAPI Integration)

Gradio provides a remarkably powerful API generation engine, but its default `gr.Blocks` or `gr.Interface` deployments bundle substantial amounts of built-in Svelte-based frontend code. To eliminate this overhead, save megabytes of space, and gain complete control over the React UI, Gradio must be utilized strictly in **Server mode**.

By instantiating the `gradio.Server` class directly, the backend acts purely as a headless FastAPI instance powered by Gradio's underlying inference and queuing engine.⁵⁰ This grants all the backend benefits of Gradio—such as robust request queuing (which prevents server timeouts if multiple users upload images simultaneously), streaming capabilities, and multipart form handling—without forcing the download of unnecessary UI libraries.⁵⁰

Python

```
# backend/main.py
import os
from gradio import Server
from fastapi.staticfiles import StaticFiles
from fastapi.responses import FileResponse
import onnxruntime as ort
import numpy as np
from PIL import Image

# Initialize the lightweight ONNX runtime session
session = ort.InferenceSession("model_int8.onnx")

# Instantiate the headless Gradio Server (which is a FastAPI app)
app = Server()
```

```

@app.api(name="predict_body_type")
def predict(image_path: str) -> dict:
    # 1. Load and preprocess image using PIL and numpy
    img = Image.open(image_path).resize((224, 224))
    img_data = np.array(img).astype(np.float32) / 255.0
    img_data = np.transpose(img_data, (2, 0, 1)) # HWC to CHW
    img_data = np.expand_dims(img_data, axis=0) # Add batch dimension

    # 2. Execute rapid CPU inference via ONNX
    outputs = session.run(None, {"input": img_data})
    probabilities = softmax(outputs)

    # 3. Map to the 8 specific classes
    classes =

    return {classes[i]: float(probabilities[i]) for i in range(8)}

```

Minimizing the React Frontend Payload

A standard React application generated via create-react-app or older bundlers can easily exceed 30-40 MB in development mode due to the sheer size of the node_modules dependency tree.⁵² To fit within the 95 MB total project limit alongside the ONNX model and Python environment, the frontend must be aggressively optimized into a tiny, static production build.

The React frontend should be constructed and compiled using a modern, lightning-fast bundler like **Vite**. The following optimization techniques are absolutely mandatory:

1. **Strict Dependency Management:** Avoid importing massive monolithic libraries like lodash or heavy component libraries unless absolutely necessary. Use native JavaScript functions and lightweight CSS modules.⁵³ Every package is a future liability against the 95 MB limit.³⁵
2. **Tree-Shaking and Code Splitting:** Vite utilizes Rollup under the hood, which performs static analysis of ES modules to achieve tree-shaking, effectively stripping out any unused code from third-party libraries.⁵³ Configure the bundler to split the application into minimal chunks using dynamic imports (React.lazy) for distinct routes.⁵³
3. **Production Build Compilation:** Running npm run build minifies the HTML, CSS, and JavaScript, stripping out all developer tools, warnings, and source maps. A properly optimized React SPA should compile to a dist folder no larger than 2 to 5 MB.⁵³

To connect the React interface to the headless Gradio backend without importing heavy REST clients like Axios, utilize the highly lightweight @gradio/client library. This JavaScript client enables direct, optimized WebSocket and HTTP communication with the API endpoints defined

in the Python server.⁵⁰

JavaScript

```
// frontend/src/api.js
import { Client, handle_file } from "@gradio/client";

export async function classifyCar(imageFile) {
  try {
    // Connect directly to the local Gradio Server endpoint
    const app = await Client.connect("http://localhost:7860");

    // Pass the image file to the predict_body_type API
    const result = await app.predict("/predict_body_type", {
      image_path: handle_file(imageFile)
    });

    return result.data; // Returns the dictionary of class probabilities
  } catch (error) {
    console.error("Inference Error:", error);
    throw error;
  }
}
```

Unified Delivery via FastAPI

To submit a single, cohesive project folder to the university's evaluation system, it is highly impractical to require the evaluators to boot up a separate Node.js server for the frontend and a Python server for the backend.⁶¹ Instead, the architecture must be unified. The Python backend must be configured to serve the compiled React frontend directly alongside the API.

Once the React application is built into the dist directory, standard FastAPI routing can be appended to the gradio.Server instance. Using FastAPI's StaticFiles, the dist folder is mounted to the application. A "catch-all" route is then added to ensure that any navigation within the React application defaults back to index.html, allowing React Router to handle the page state seamlessly without triggering 404 errors from the Python server.⁶²

Python

```

# backend/main.py (Continued)

# Ensure the React build uses relative paths (base: './' in vite.config.js)
dist_dir = os.path.abspath(os.path.join(os.path.dirname(__file__), "../frontend/dist"))

# Mount the static assets directory (CSS, JS, Images) generated by Vite
app.mount("/assets", StaticFiles(directory=f"{dist_dir}/assets"), name="assets")

# Catch-all route to serve the React Single Page Application
@app.get("/{full_path:path}")
async def serve_frontend(full_path: str):
    requested_file = os.path.join(dist_dir, full_path)

    # Serve specific static files if they explicitly exist (e.g., favicon.ico)
    if os.path.isfile(requested_file):
        return FileResponse(requested_file)

    # Fallback to index.html to allow React Router to manage the UI state
    return FileResponse(os.path.join(dist_dir, "index.html"))

# Launch the unified server
if __name__ == "__main__":
    app.launch(server_port=7860, prevent_thread_lock=True)

```

By unifying the architecture in this precise manner, the entire directory structure remains impeccably clean and portable. The Python virtual environment can be stripped of massive deep learning frameworks by explicitly generating a requirements.txt that only contains onnxruntime, fastapi, and gradio.³⁵ The FastViT or MobileNetV4 model is minimized to under 10 MB via dynamic ONNX quantization, and the React frontend is a lightweight 3 MB static bundle.

This holistic, end-to-end engineering approach guarantees that the final submission will effortlessly fit beneath the strict 95 MB constraint. It mitigates domain bias through strategic background processing, resolves fine-grained morphological confusion via Knowledge-Distillation Label Smoothing, and provides a remarkably fast, highly accurate, and visually polished application ready for live evaluation on standard CPU hardware.